



Grafos p2

Programación 3 TUDAI

Cursada 2026

Docente: Federico Casanova

Recorridos sobre un Grafo

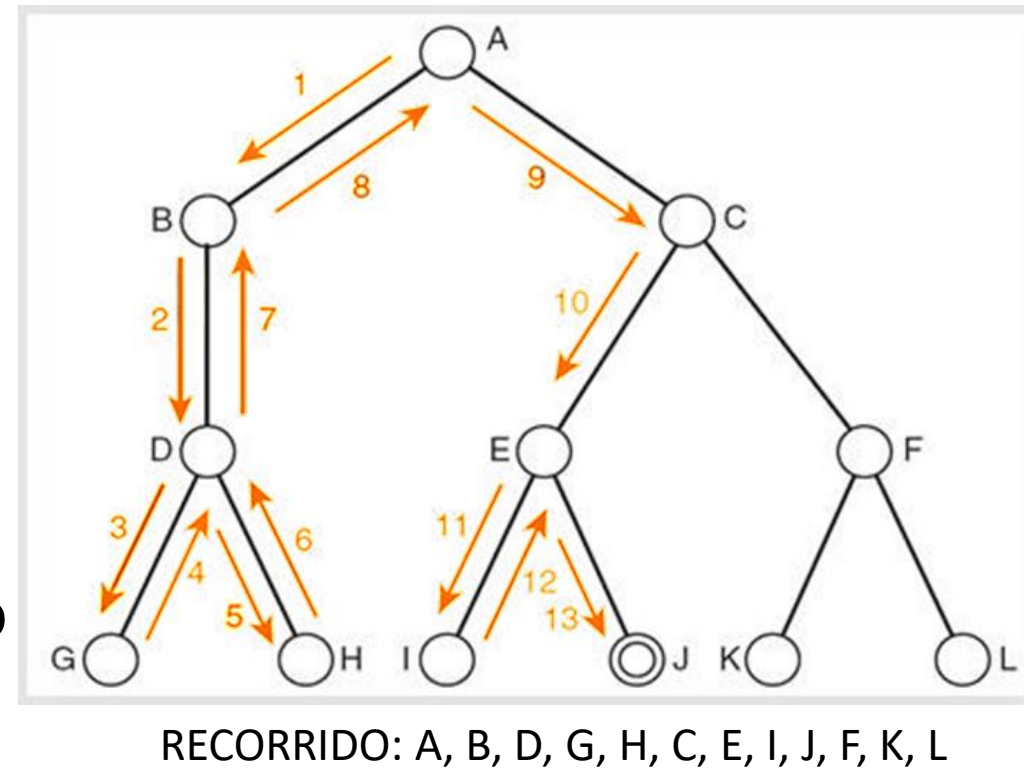
Un recorrido (o exploración) de un Grafo visita **todos** los vértices en una secuencia determinada.

Vamos a ver dos de los recorridos más comunes de un grafo, el recorrido en profundidad (DFS) y el recorrido en amplitud (BFS).

DFS (Depth-First Search)

Recorrido en profundidad

- Es una generalización del recorrido preorden de un árbol.
- Al tratarse de Grafos, ya no tenemos hijos pero tenemos nodos adyacentes de un vértice.
- Recursivamente visita todos los vértices adyacentes no visitados de un vértice. **Hay que llevar registro de cuáles se visitaron para no volver a visitar (varias opciones).**
- Se debe tener cuidado porque en un grafo se puede llegar a un vértice por mas de un camino, por eso hay que visitar sólo los nodos que aun no fueron visitados por el recorrido.



DFS: Pseudo-Código

```
DFS(G)
{
    for each vértice u ∈ G.vertices()
    {
        u.color = BLANCO;
    }

    for each vértice u ∈ G.vertices()
    {
        if (u.color == BLANCO)
            DFS_Visit(u);
    }
}
```

Registro de vértices no visitados:

- Opción más sencilla, estructura que almacene para cada vértice si está visitado o no (true/false) o sea dos estados posibles. Se inicializan todos en false.
- En el ejemplo que veremos se usa otro enfoque, se almacena en una estructura una marca para cada vértice que tiene 3 estados posibles, denotados por un color: BLANCO, AMARILLO o NEGRO. El BLANCO sería como el no visitado, y distinto a BLANCO (o sea AMARILLO o NEGRO) serían visitados. Pero esta separación de colores nos permitirá hacer otras cosas, como ya veremos.

DFS: Pseudo-Código

```
DFS(G)
{
    for each vértice u ∈ G.vertices()
    {
        u.color = BLANCO;
    }

    for each vértice u ∈ G.vertices()
    {
        if (u.color == BLANCO)
            DFS_Visit(u);
    }
}
```

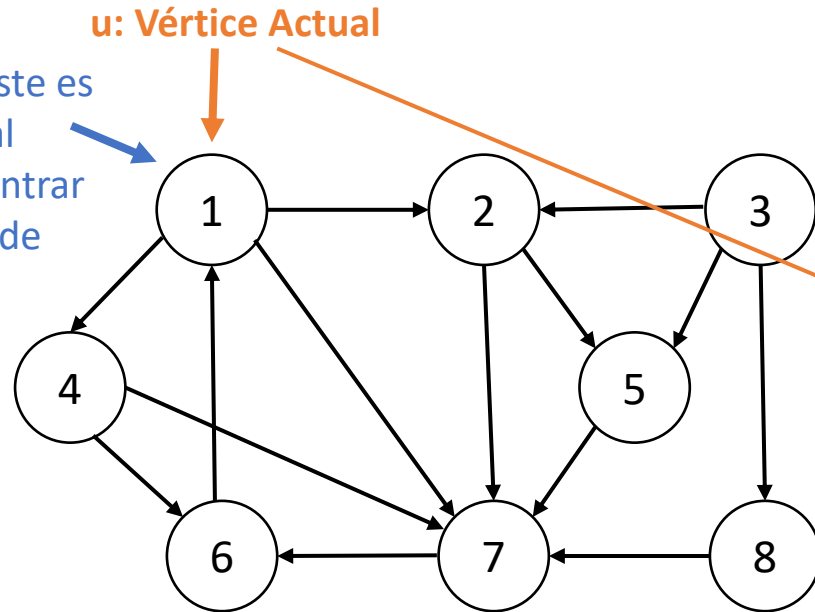
```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

Es un planteo recursivo.

DFS: Secuencia de ejemplo

Supongamos este es
el Vértice Inicial
que agarró al entrar
al segundo for de
DFS(G)



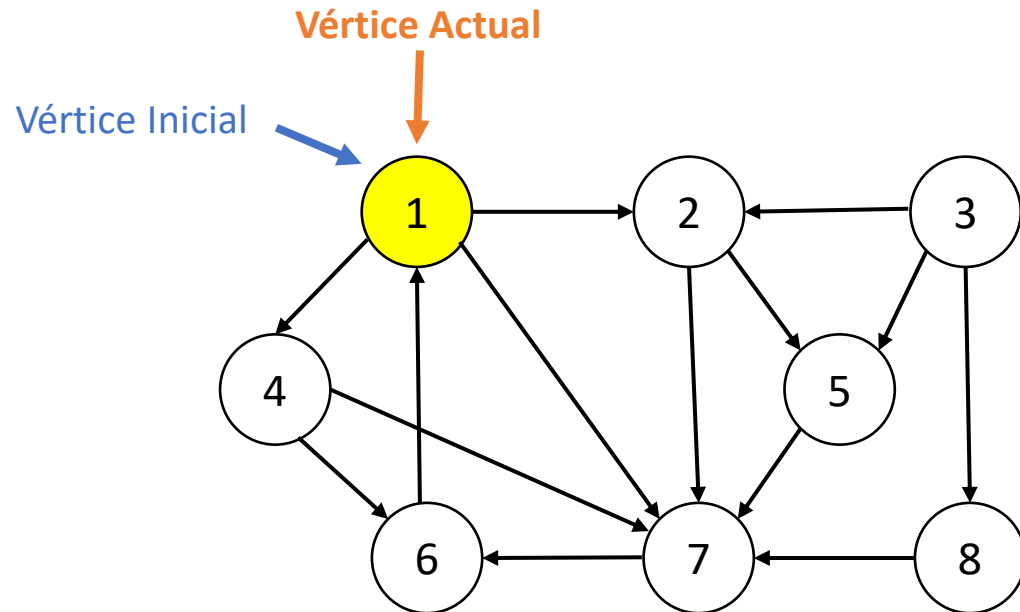
DFS(G)

```
{  
  for each vértice u ∈ G.vertices()  
  {  
    u.color = BLANCO;  
  }  
  
  for each vértice u ∈ G.vertices()  
  {  
    if (u.color == BLANCO)  
      DFS_Visit(u);  
  }  
}
```

DFS_Visit(u)

```
{  
  u.color = AMARILLO;  
  for each v ∈ u.adyacentes()  
  {  
    if (v.color == BLANCO)  
      DFS_Visit(v);  
  }  
  
  u.color = NEGRO;  
}
```

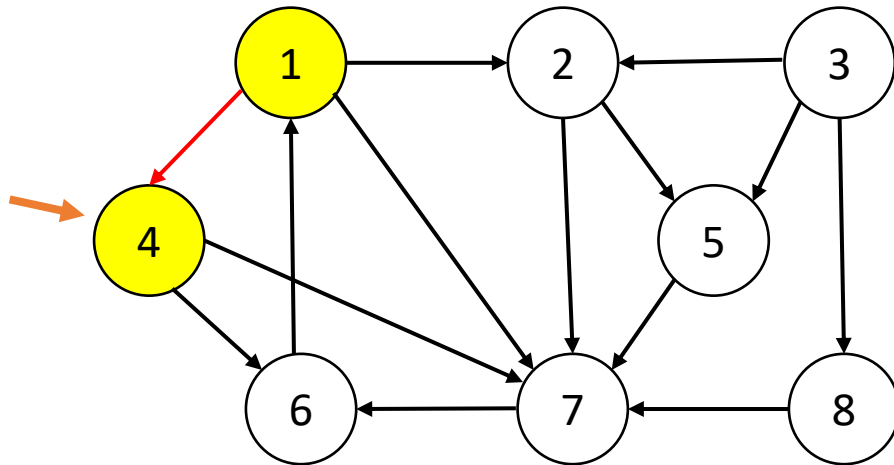
DFS: Secuencia de ejemplo



```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

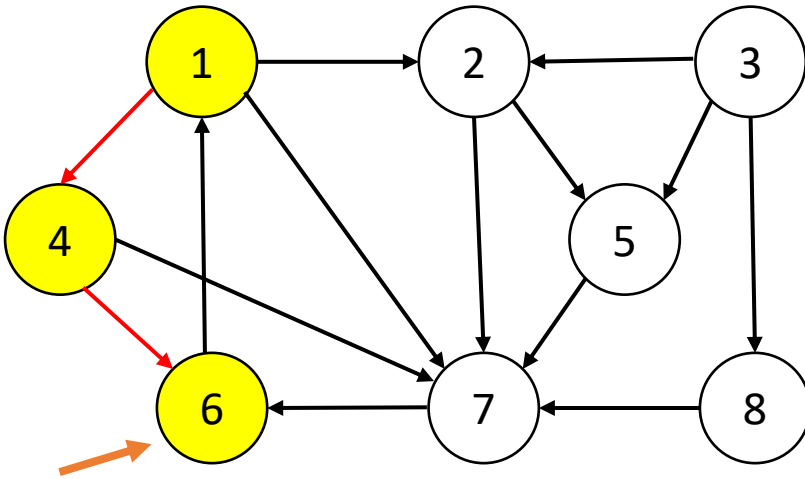
DFS: Secuencia de ejemplo



```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

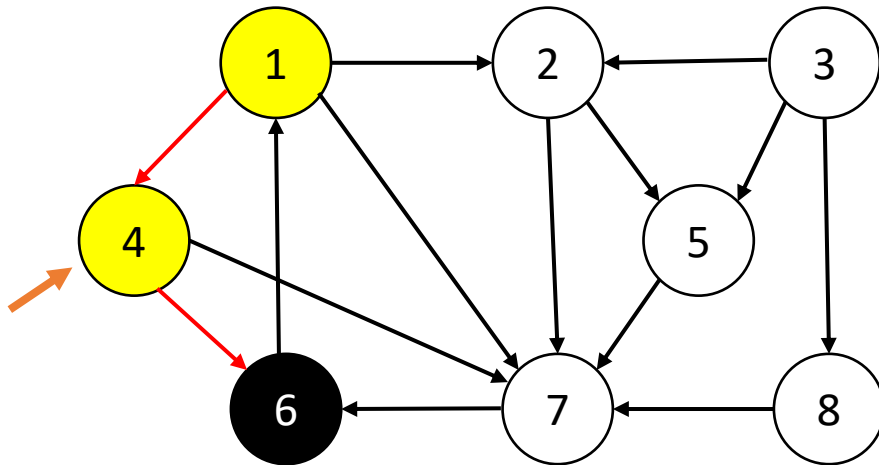

DFS: Secuencia de ejemplo



```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

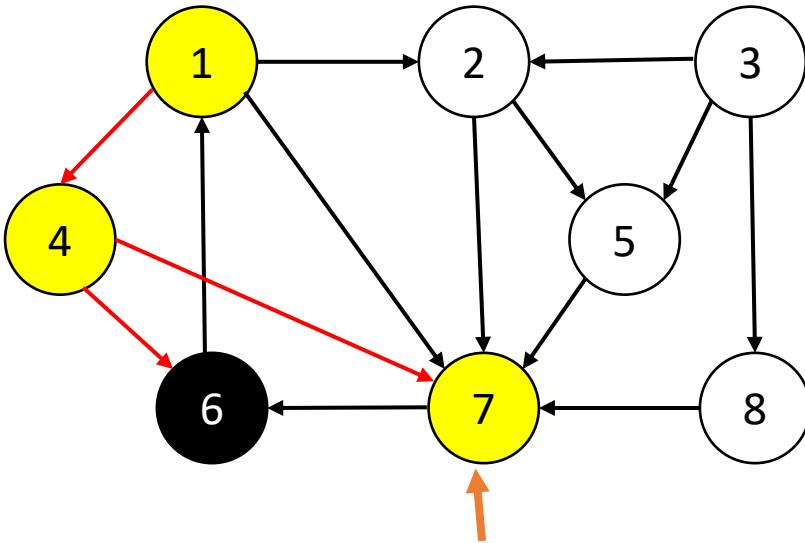
DFS: Secuencia de ejemplo



```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

DFS: Secuencia de ejemplo

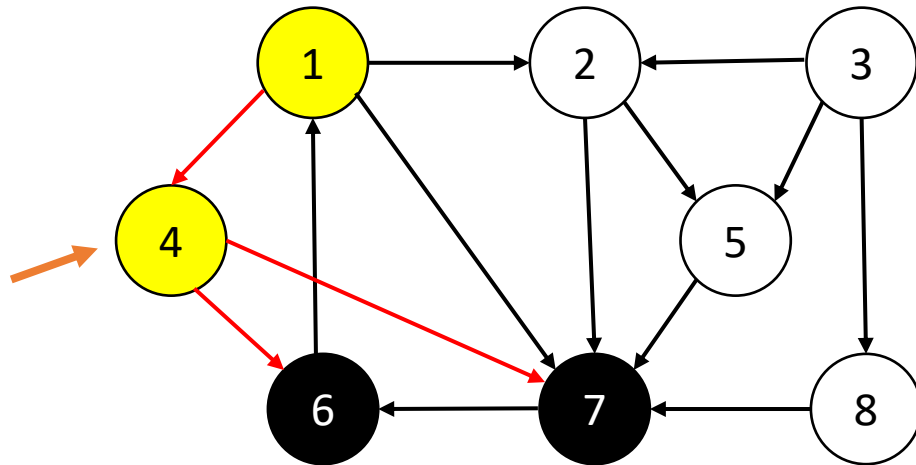


```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

En todo momento los vértices AMARILLO muestran el camino actual “activo” desde el vértice origen hasta el vértice en el cual se encuentra la recursión.

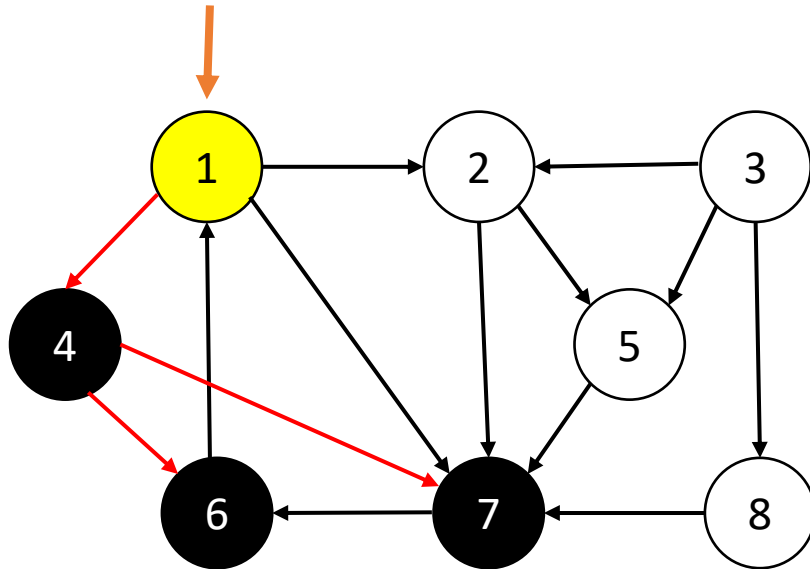
DFS: Secuencia de ejemplo



```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

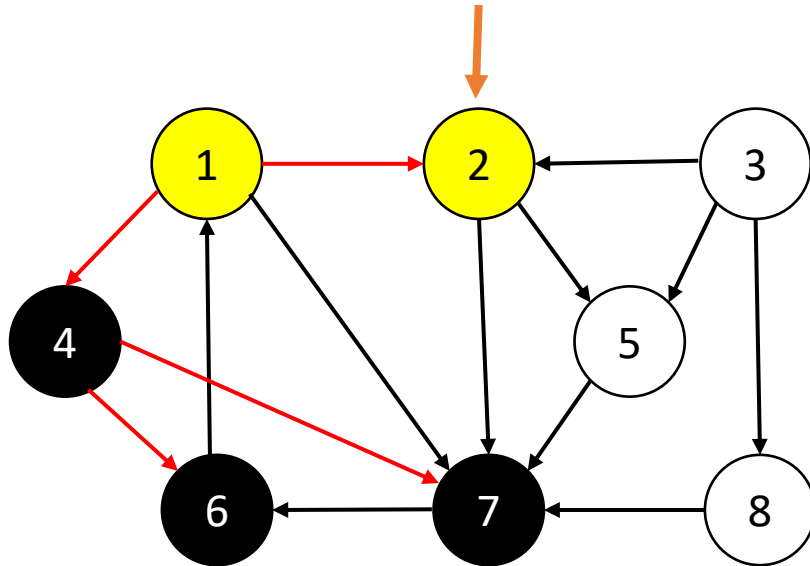
DFS: Secuencia de ejemplo



```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

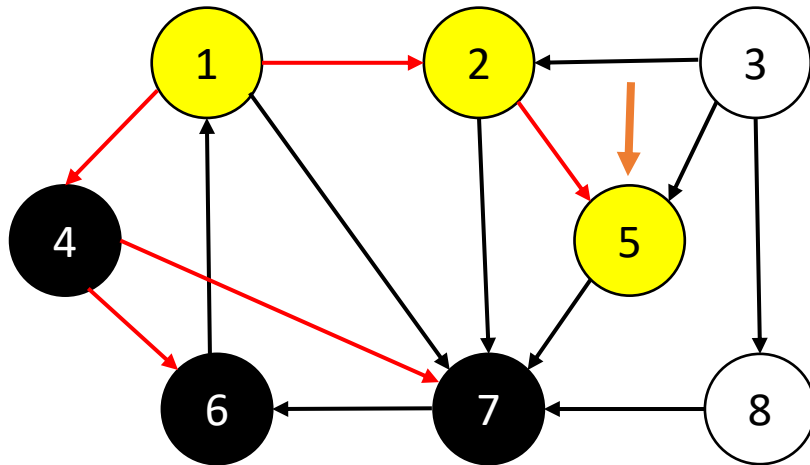
DFS: Secuencia de ejemplo



```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

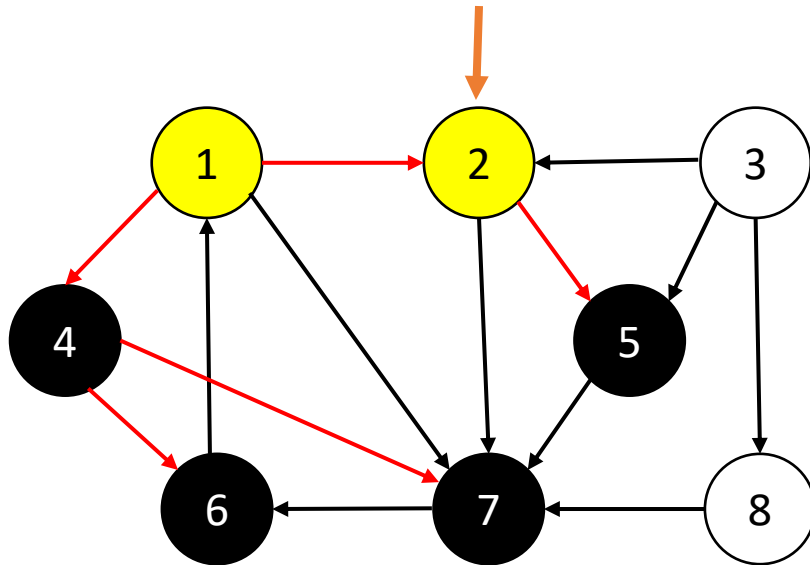
DFS: Secuencia de ejemplo



```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

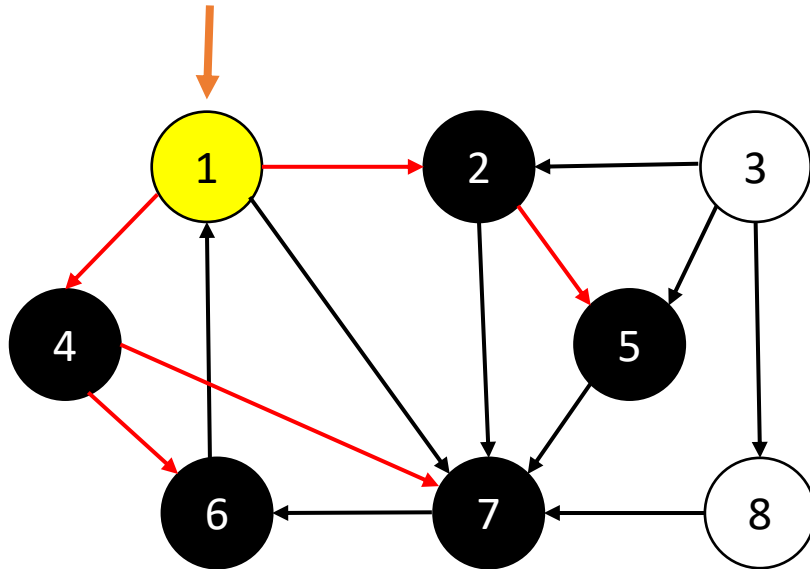

DFS: Secuencia de ejemplo



```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

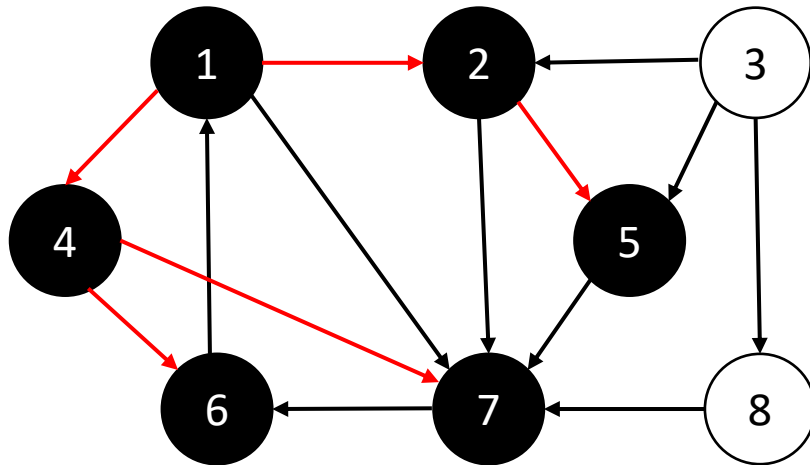
DFS: Secuencia de ejemplo



```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

DFS: Secuencia de ejemplo



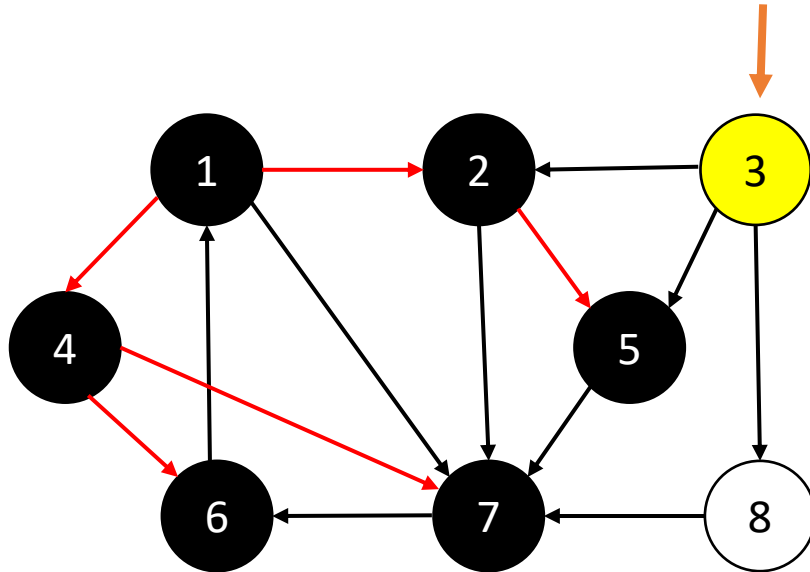
DFS(G)

```
{  
  for each vértice u ∈ G.vertices()  
  {  
    u.color = BLANCO;  
  }  
  
  for each vértice u ∈ G.vertices()  
  {  
    if (u.color == BLANCO)  
      DFS_Visit(u);  
  }  
}
```

DFS_Visit(u)

```
{  
  u.color = AMARILLO;  
  for each v ∈ u.adyacentes()  
  {  
    if (v.color == BLANCO)  
      DFS_Visit(v);  
  }  
  
  u.color = NEGRO;  
}
```

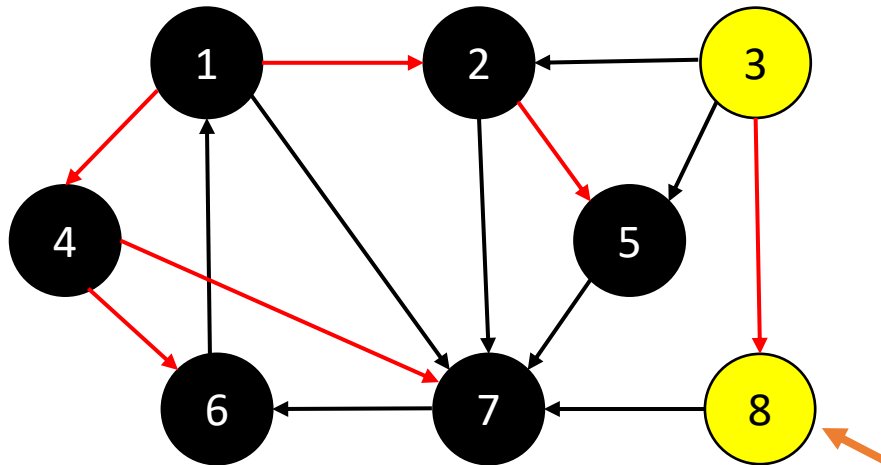
DFS: Secuencia de ejemplo



```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

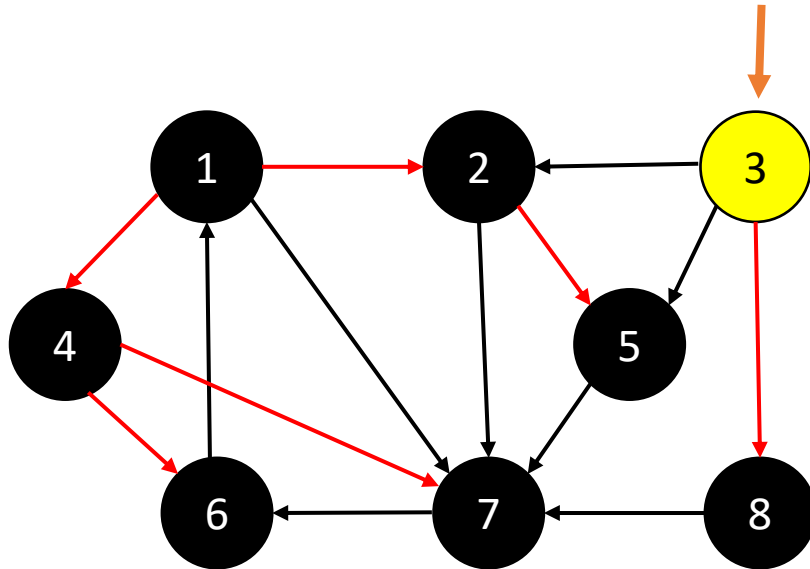
DFS: Secuencia de ejemplo



```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

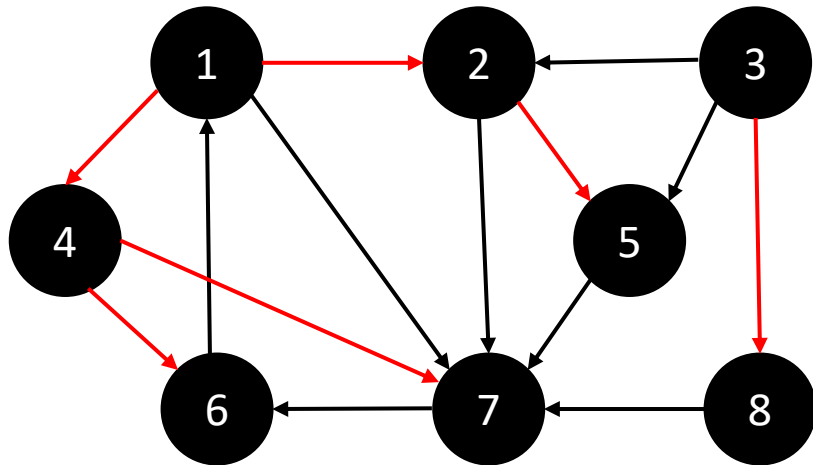
DFS: Secuencia de ejemplo



```
DFS_Visit(u)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v);
    }

    u.color = NEGRO;
}
```

DFS: Secuencia de ejemplo



DFS(G)

```
{  
  for each vértice u ∈ G.vertices()  
  {  
    u.color = BLANCO;  
  }  
  
  for each vértice u ∈ G.vertices()  
  {  
    if (u.color == BLANCO)  
      DFS_Visit(u);  
  }  
}
```

DFS_Visit(u)

```
{  
  u.color = AMARILLO;  
  for each v ∈ u.adyacentes()  
  {  
    if (v.color == BLANCO)  
      DFS_Visit(v);  
  }  
  
  u.color = NEGRO;  
}
```

DFS

- El orden en que se toman los vértices al llamar a $\text{DFS}(G)$ **puede ser cualquiera.**
- El orden en que se toman los adyacentes de un vértice **puede ser cualquiera.**
- Dependerá del orden en que fueron cargados vértices y arcos en el grafo.
- Entonces: puede haber **muchos recorridos DFS posibles y correctos** desde un mismo origen.

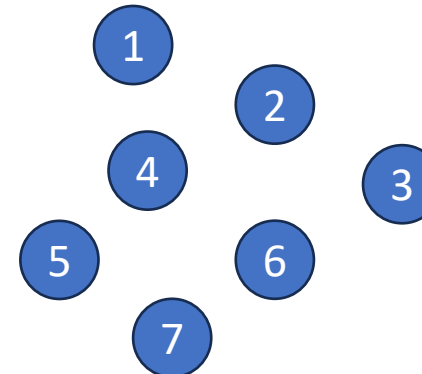
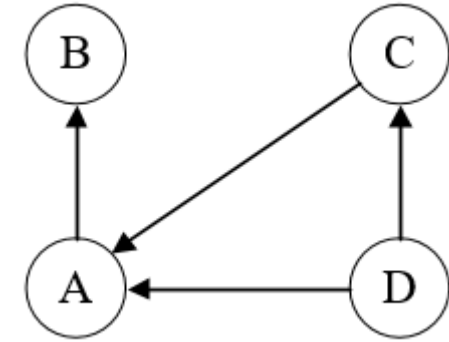
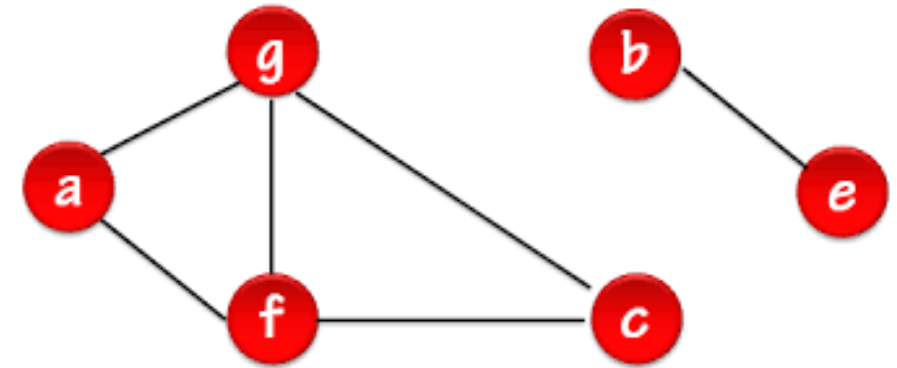
El DFS siempre visita a todos los vértices sin importar la forma ni el tipo del grafo.

```
DFS(G)
{
  for each vértice u ∈ G.vertices()
  {
    u.color = BLANCO;
  }

  for each vértice u ∈ G.vertices()
  {
    if (u.color == BLANCO)
      DFS_Visit(u);
  }
}
```

```
DFS_Visit(u)
{
  u.color = AMARILLO;
  for each v ∈ u.adyacentes()
  {
    if (v.color == BLANCO)
      DFS_Visit(v);
  }

  u.color = NEGRO;
}
```

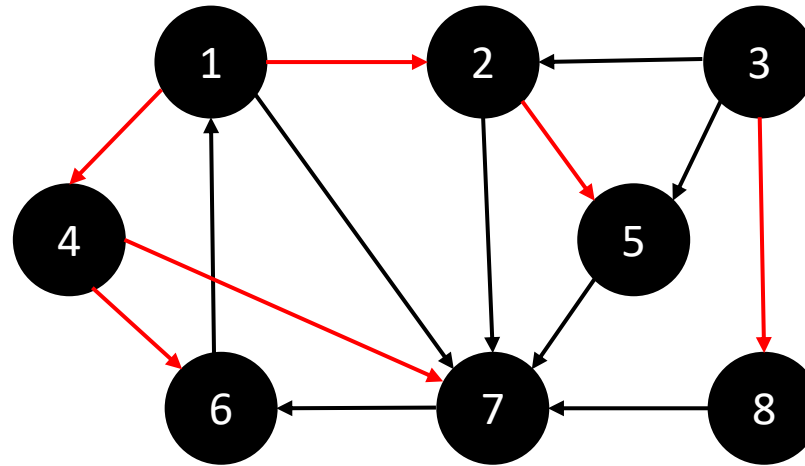


DFS: Clases de Arcos

DFS introduce una distinción importante entre arcos en el grafo original:

- *Arcos Tree* : encuentra vértices nuevos (blancos)
- Los arcos tree forman un bosque abarcador

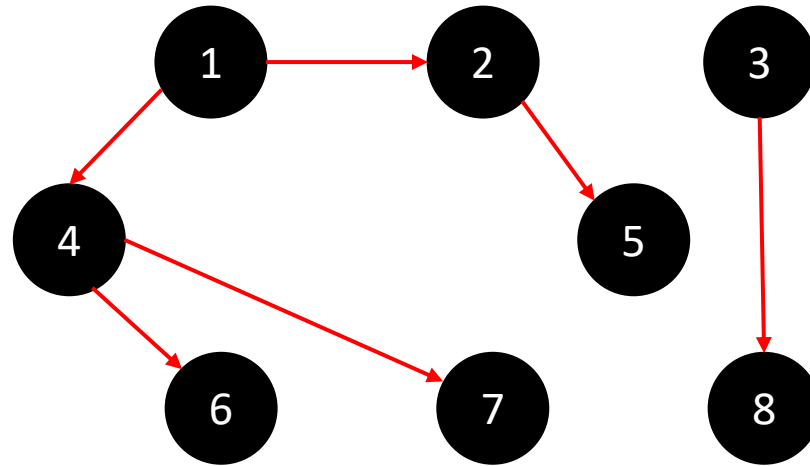
DFS Clasificación de Arcos: Tree



**Arcos Tree de ese recorrido
DFS (arcos rojos)**

También llamados arcos de árbol.

DFS Clasificación de Arcos: Tree



Arcos Tree

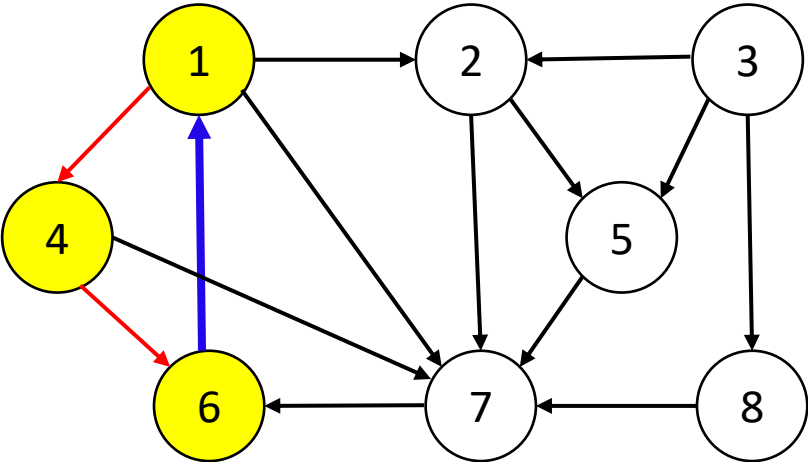
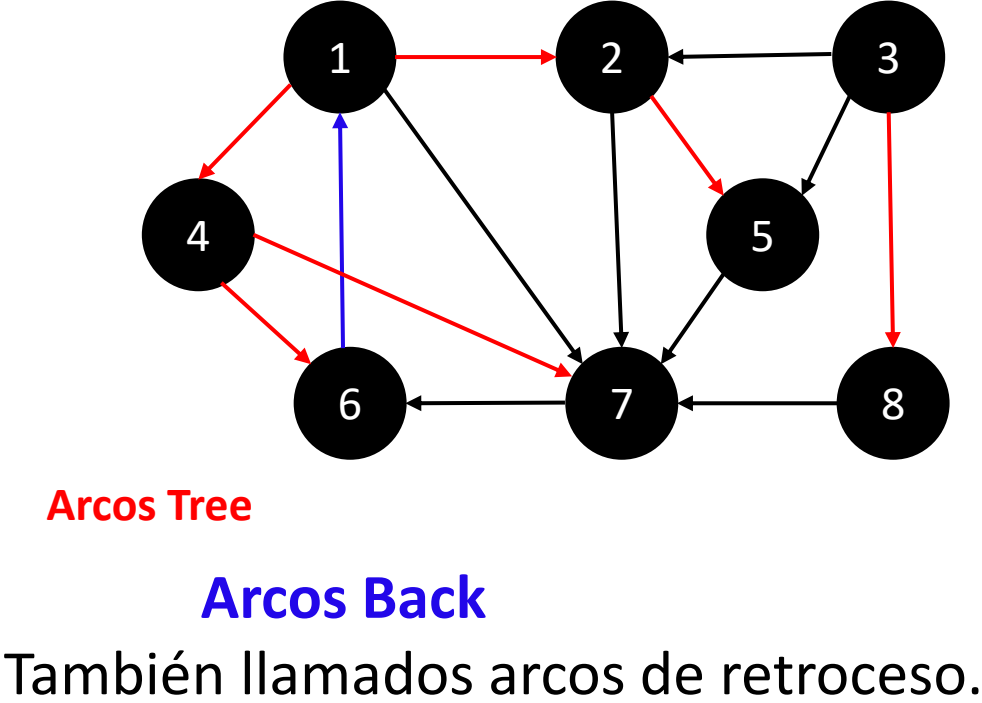
Los arcos tree forman el llamado “bosque abarcador en profundidad”.

DFS: Clases de arcos

DFS introduce una distinción importante entre arcos en el grafo original:

- *Arcos Back*: desde el descendiente al ancestro
- Encuentra un vértice amarillo (de amarillo a amarillo)

DFS Clasificación de Arcos: Back



DFS y Ciclos en los grafos

Un Grafo es acíclico si y sólo si en el recorrido DFS no se encuentran arcos de tipo "back". O sea nunca se intenta ir de un vértice amarillo a otro amarillo.

Podemos usar el recorrido DFS entonces para hacer lo que se conoce como test de aciclicidad de un grafo, que es determinar si el grafo es cíclico o acíclico.

DFS: ¿ Cómo modificarlo para detectar ciclos ?

```
DFS(G)
{
  for each vértice u ∈ G.vertices()
  {
    u.color = BLANCO;
  }

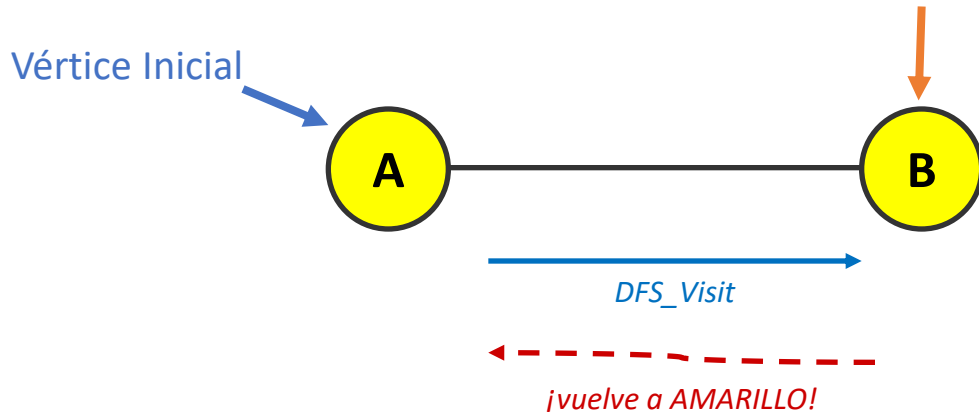
  for each vértice u ∈ G.vertices()
  {
    if (u.color == BLANCO)
      DFS_Visit(u);
  }
}
```

```
DFS_Visit(u)
{
  u.color = AMARILLO;
  for each v ∈ u.adyacentes()
  {
    if (v.color == BLANCO)
      DFS_Visit(v);
  }
  u.color = NEGRO;
}
```

 **else**
if (v.color == AMARILLO)
print("Hay Ciclo!");

DFS Problema con ciclos en grafos no dirigidos

GND ejemplo:



1. DFS_Visit(A): A pasa a **AMARILLO**

2. A tiene adyacente B, entonces llama a DFS_Visit(B)

3. B pasa a **AMARILLO**, recorre adyacentes...

4. Entre los adyacentes de B estará A (ya que A es adyacente a B) y A está con color **AMARILLO** → Entonces Detecta un Ciclo Incorrecto (Recordar que un ciclo tiene un camino de arcos NO REPETIDOS).

⚠ El problema: En grafos no dirigidos, el arco {A,B} aparece en la lista de adyacencia de A y **también** en la de B. Al ir de A→B, B vuelve a ver a A por el mismo arco, y como A está AMARILLO, reporta un ciclo que no existe.

DFS Problema con ciclos en grafos no dirigidos

Posible Solución:

1. Nuevo parámetro en DFS_Visit: padre

El vértice desde el que llegamos. La primera llamada pasa NULL.

2. Controlamos: (v != padre)

Solo reporta ciclo si el adyacente AMARILLO NO es el padre. Así ignora el arco de ida.

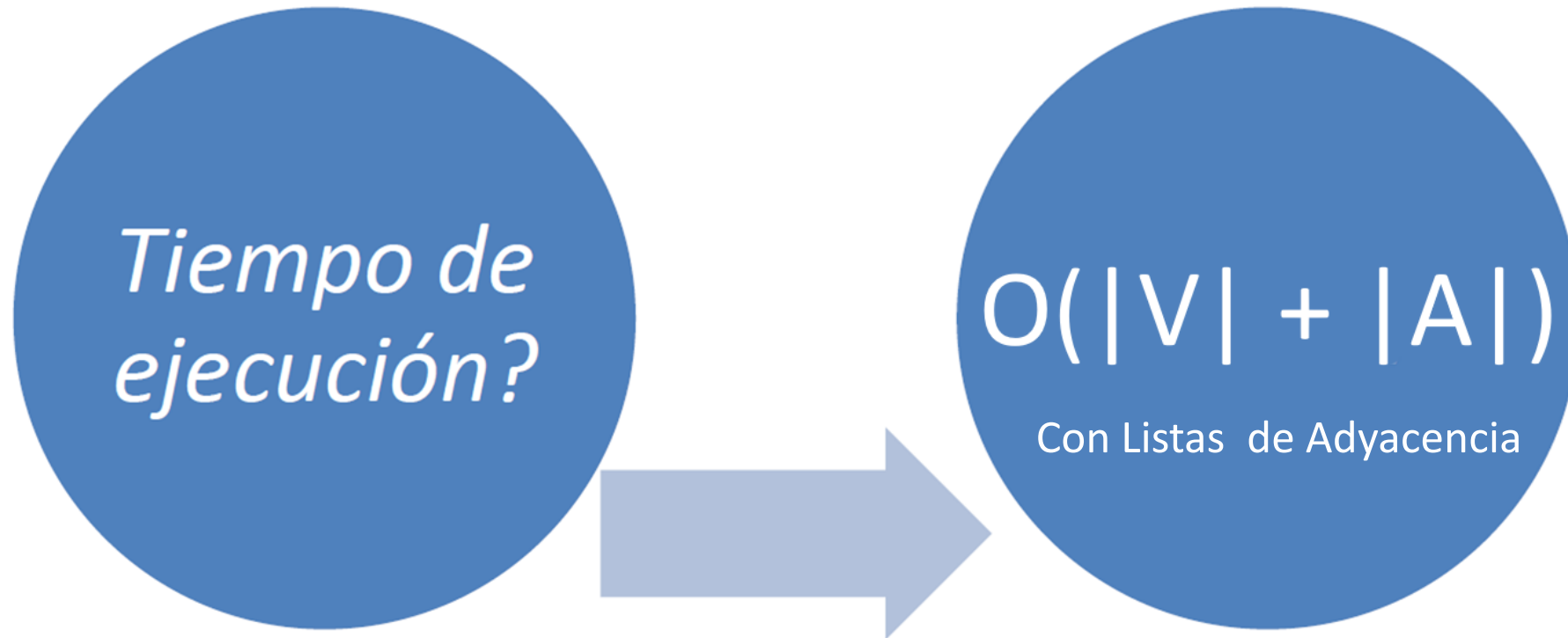
3. Llamada recursiva pasa u como padre de v

DFS_Visit(v, u) propaga el padre para que el hijo sepa de dónde vino.

En grafos dirigidos no hace falta este cambio (ya que los arcos tienen dirección).

```
DFS_Visit(u, padre)
{
    u.color = AMARILLO;
    for each v ∈ u.adyacentes()
    {
        if (v.color == BLANCO)
            DFS_Visit(v, u)
        else
            if ((v.color == AMARILLO) && (v != padre))
                print("Hay Ciclo!");
    }
    u.color = NEGRO;
}
```

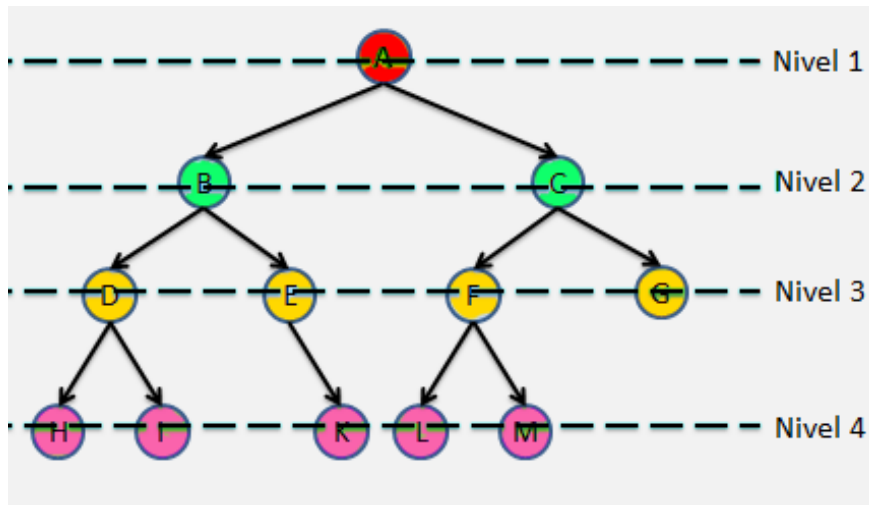
DFS: Complejidad Computacional



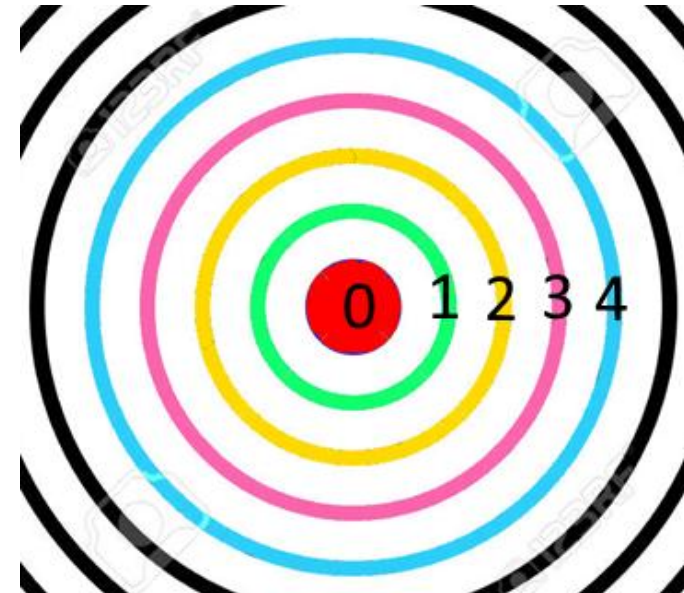
PARA UN GRAFO $G(V,A)$, EL DFS PASA UN VEZ POR CADA VERTICE, Y COMO MUCHO UNA VEZ POR CADA ARCO

BFS (Breadth-First Search): Recorrido en Amplitud

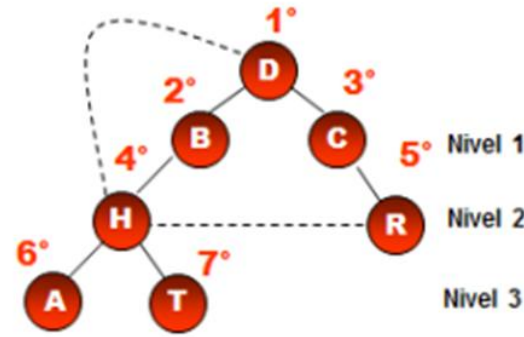
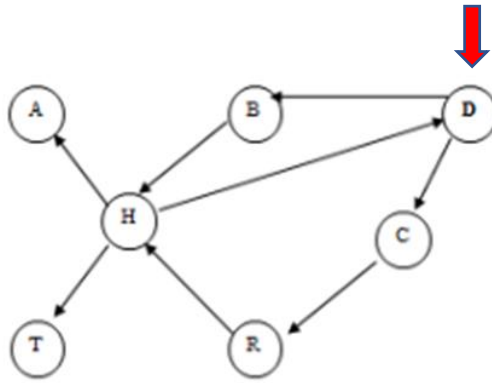
- Es una generalización de un recorrido por niveles de un árbol.



- Se realiza un recorrido por capas del grafo.
- Los nodos de una capa tienen la misma distancia al origen.



BFS



Recorrido desde Vertice $D = \{D, B, C, H, R, A, T\}$

Para hacerlo nos valemos de una estructura de fila (cola, queue):

BFS (Grafo G):

Vaciar la fila F .

Para cada vértices v de G

Marcar v como NO_VISITADO.

Para cada vértice v de G

Si v es NO_VISITADO:

BFS(G, v)

BFS (Grafo G , vértice s):

Marcar el vértice s como VISITADO.

Agregar s a la fila F .

Mientras la fila F no esté vacía

Tomamos vértice x de la fila,

Para cada vértice y adyacente a x :

Si y es NO_VISITADO :

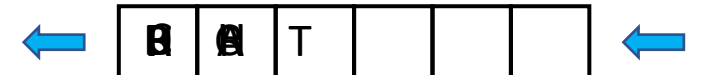
Marcar el vértice y como VISITADO.

Agregar y a la fila F .

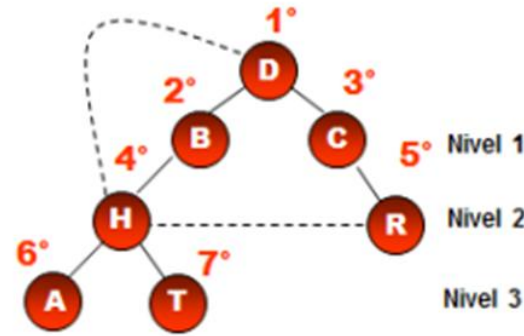
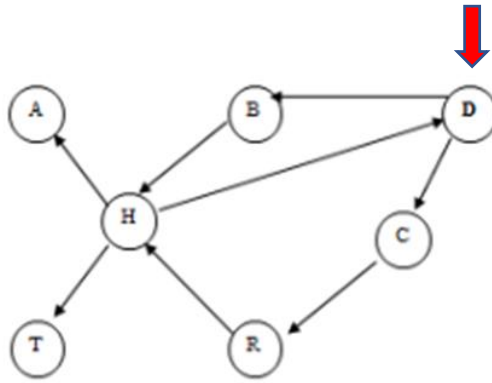
"Foto" de la fila en este punto

- 1) D
- 2) BC
- 3) CH
- 4) HR
- 5) RAT
- 6) AT
- 7) T
- 8)

Es un planteo iterativo, no recursivo.



BFS



Recorrido desde Vertice $D = \{D, B, C, H, R, A, T\}$

Para hacerlo nos valemos de una estructura de fila (cola, queue):

BFS (Grafo G):

Vaciar la fila F .

Para cada vértices v de G

Marcar v como NO_VISITADO.

Para cada vértice v de G

Si v es NO_VISITADO:

BFS(G, v)

BFS (Grafo G , vértice s):

Marcar el vértice s como VISITADO.

Agregar s a la fila F .

Mientras la fila F no esté vacía

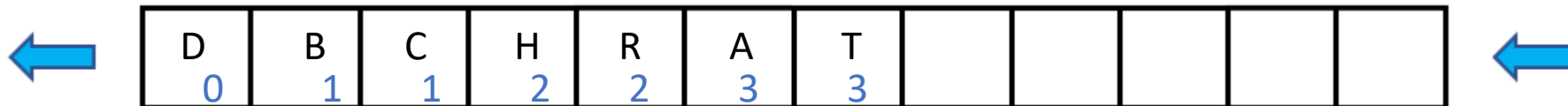
Tomamos vértice x de la fila,

Para cada vértice y adyacente a x :

Si y es NO_VISITADO :

Marcar el vértice y como VISITADO.

Agregar y a la fila F .



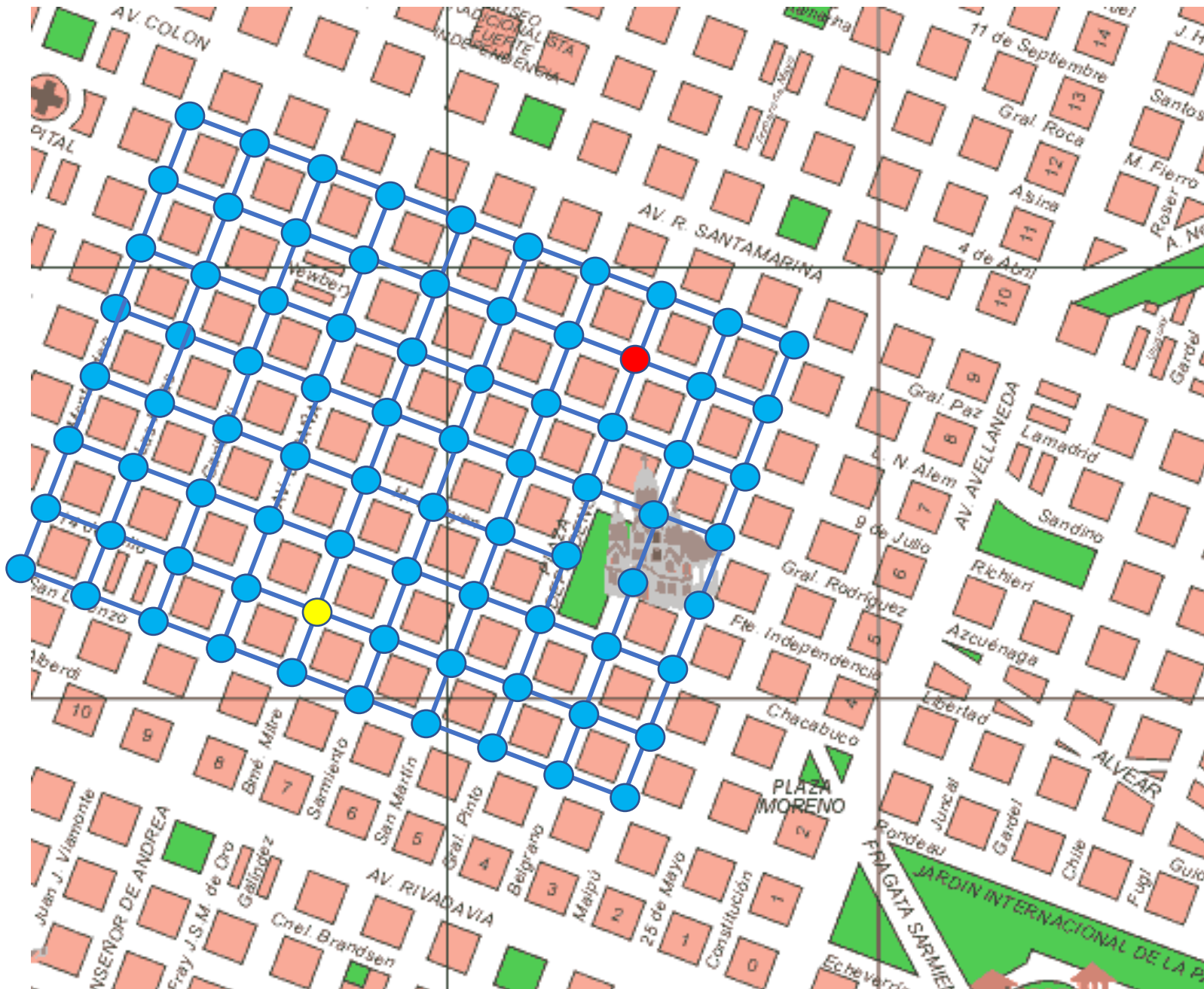
Si vemos todos los valores que entraron en la fila, el orden viene dado por el nivel.

Esto y la forma en que se agrega y saca de una fila, hacen el comportamiento del recorrido por niveles.

Problema clásico

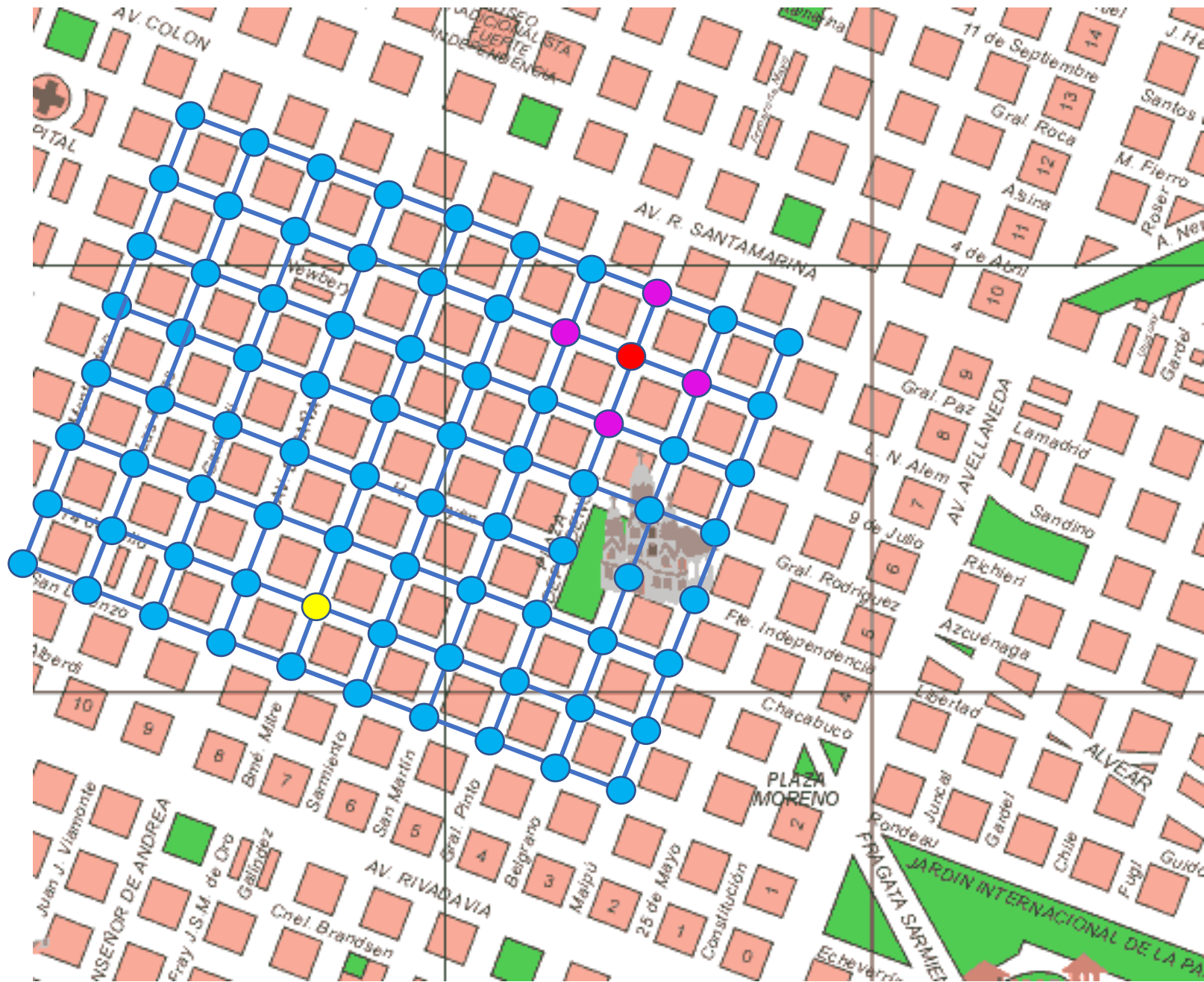
- Tenemos un Grafo no Rotulado (los arcos no tienen peso asociado). ¿Cómo encontrar el camino con menor longitud (menor cantidad de arcos) entre un vértice **v** y otro vértice **w**?
- Estrategia: Visitar primero todos los vértices de distancia 1 desde **v**, luego los de distancia 2, luego los de 3, y así sucesivamente. A medida que visitamos los vértices verificamos si encontramos el **w** que buscamos, la distancia será el nivel donde lo encontremos.
- Lo resolvemos haciendo un BFS desde **v** controlando cuando llegamos a **w**.

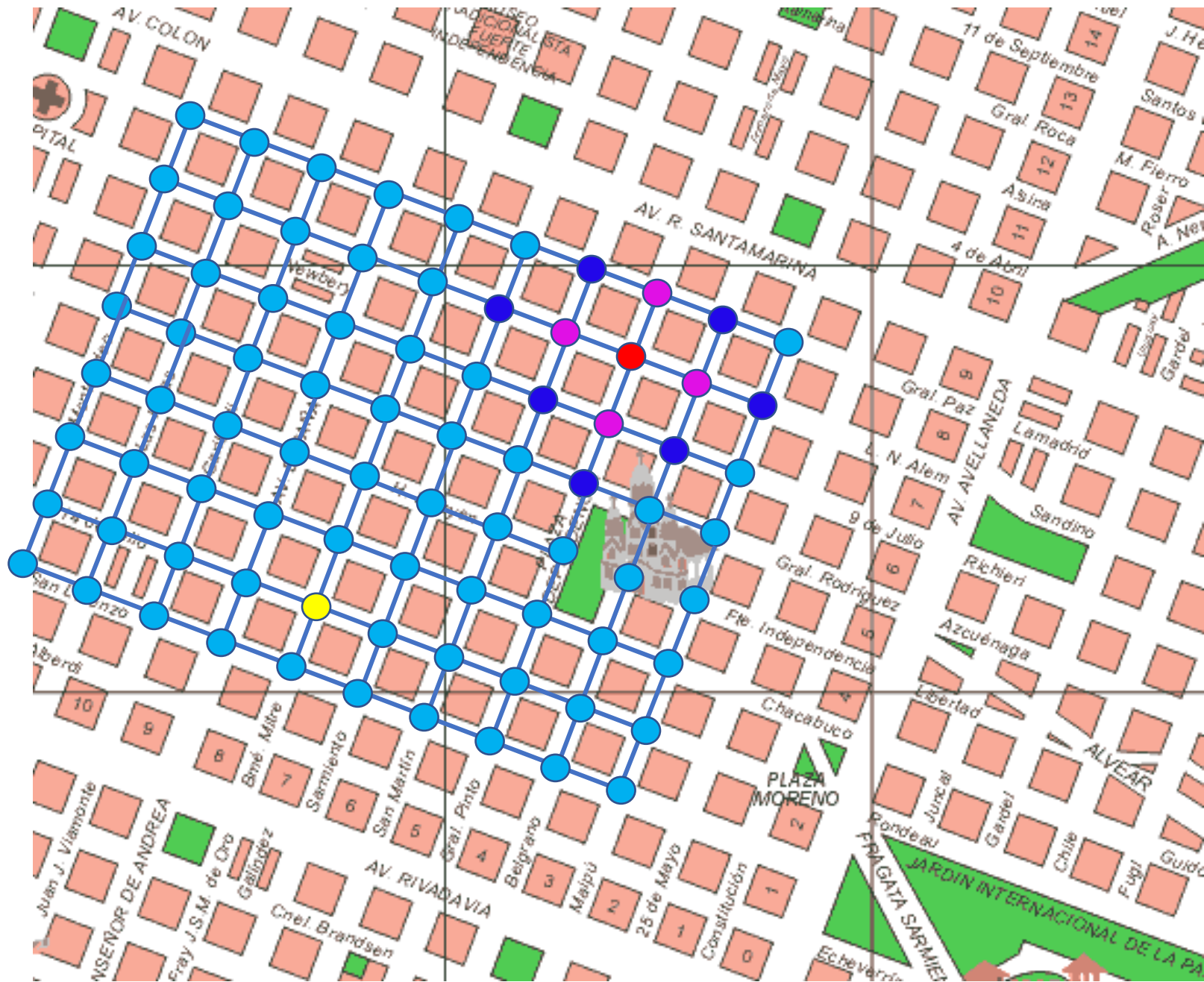


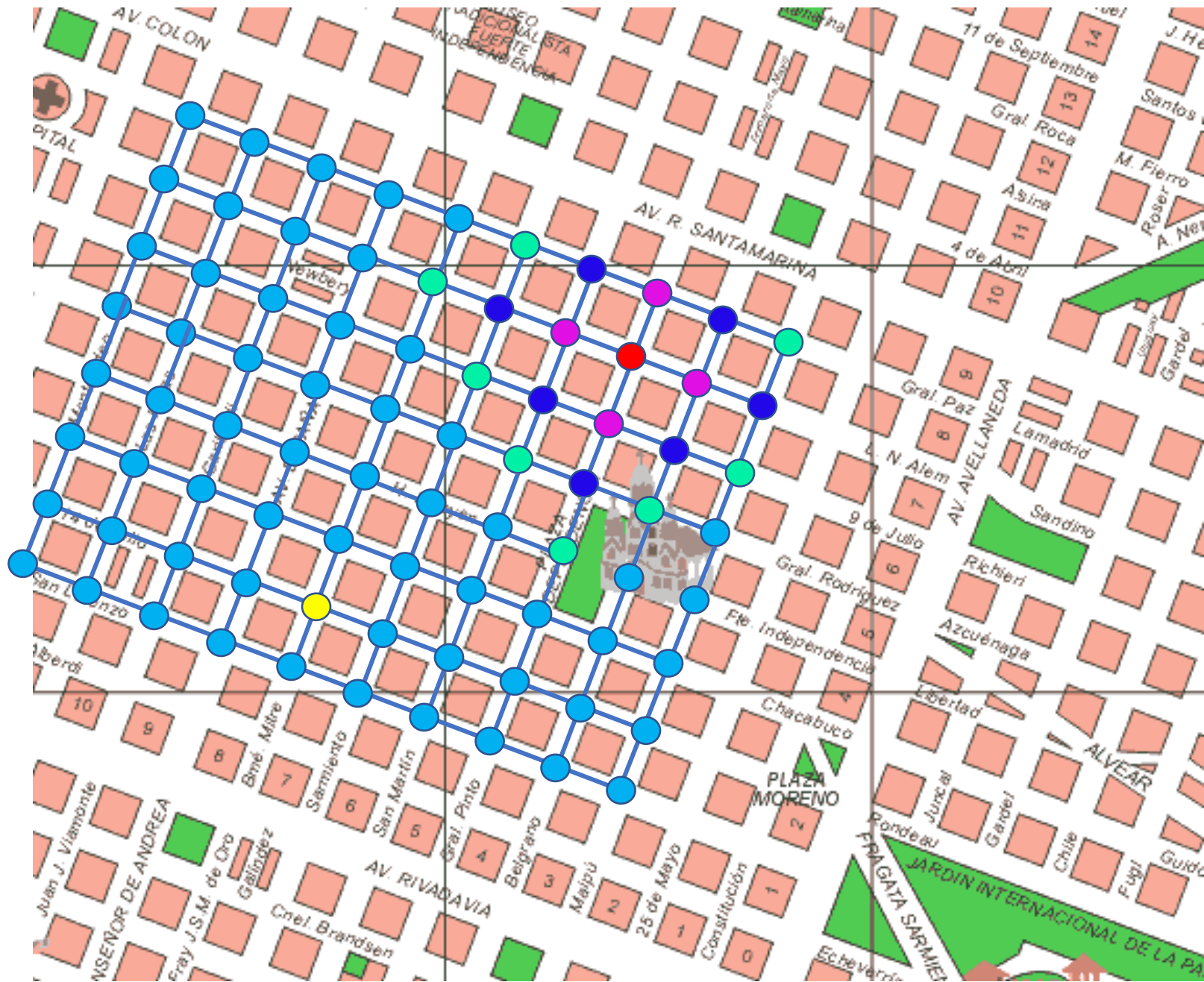


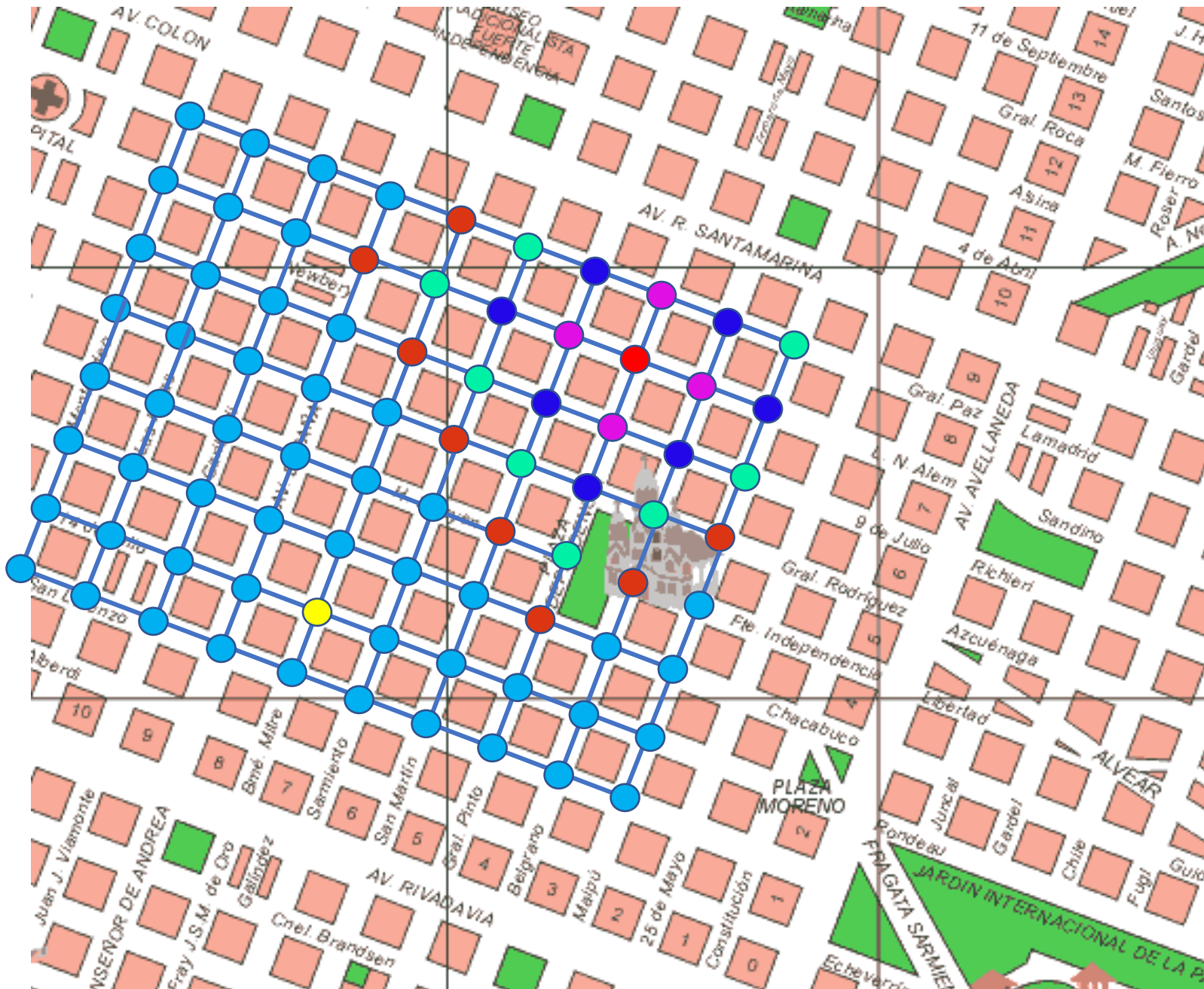
Cuál es la menor cantidad de cuadras a caminar para llegar desde rojo hasta amarillo?

Aplicamos BFS desde rojo hasta encontrar el amarillo.





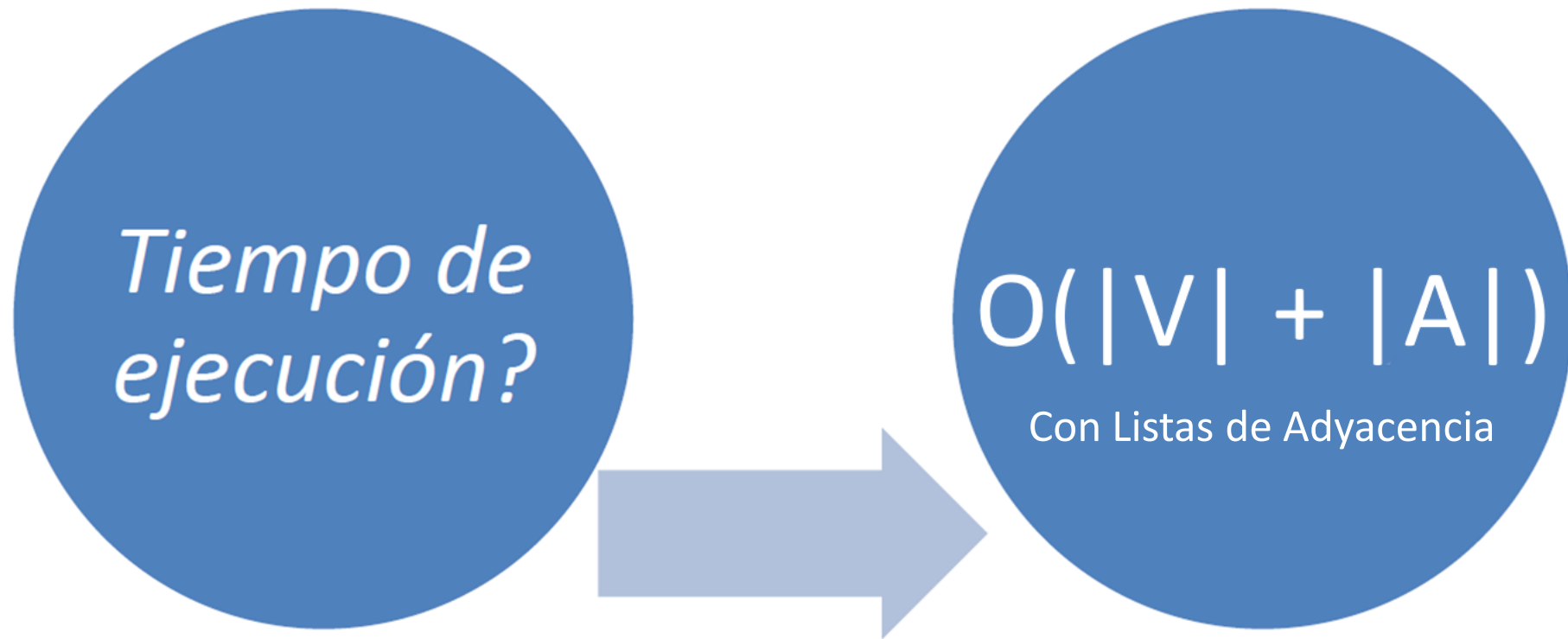




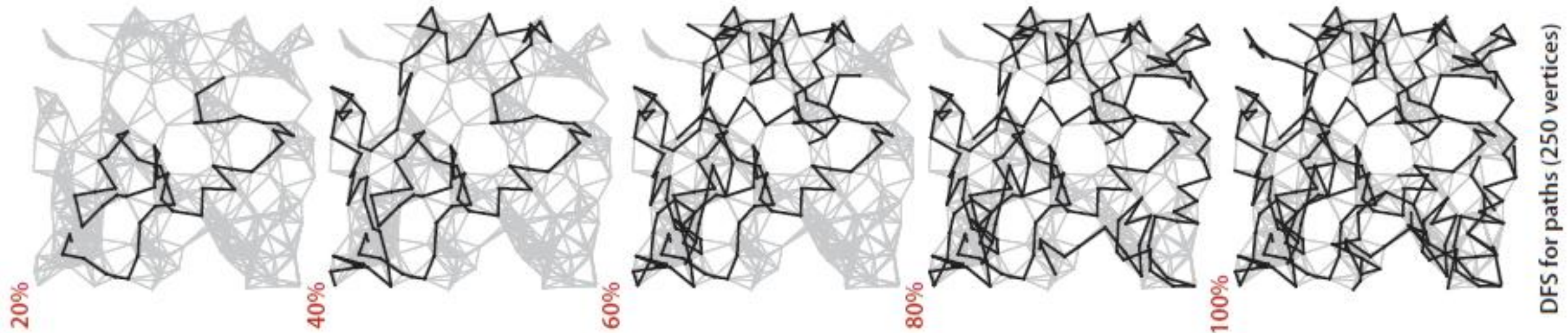
Encontramos al amarillo en el nivel 8 (distancia 8 desde el origen). O sea caminaremos como mínimo 8 cuadras.

Si el grafo fuera pesado, pero todos los pesos fueran iguales, ¿sería válido también aplicar BFS para ese problema?

BFS: Complejidad Computacional

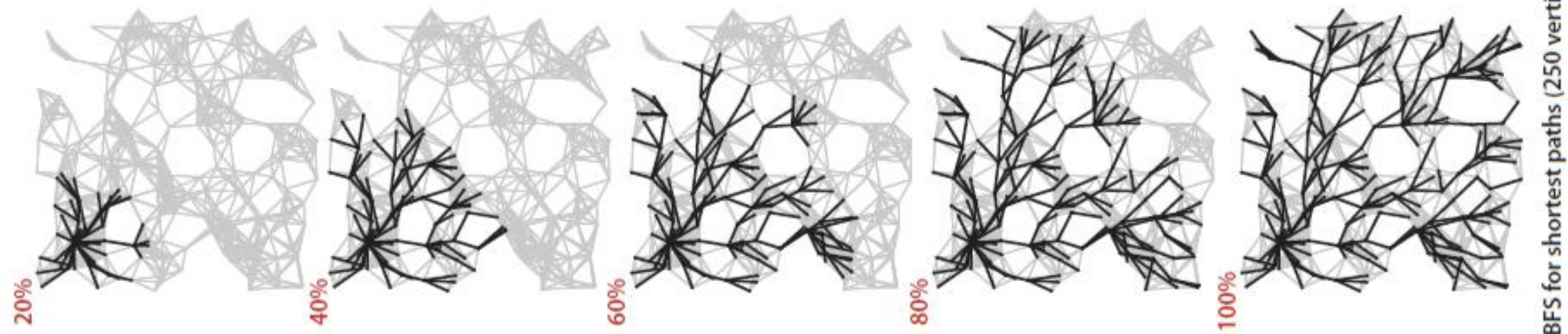


PARA UN GRAFO $G(V,A)$, EL RECORRIDO BFS PASA UN VEZ POR CADA VERTICE,
Y COMO MUCHO UNA VEZ POR CADA ARCO



DFS

Imagen del libro Introduction to Algorithms – Cormen, Leiserson, y otros.



BFS